

Unit 4 - Image Segmentation

Image segmentation is a crucial image processing technique that partitions a digital image into distinct regions or sets of pixels to simplify representation and facilitate object identification.

Point, Line & Edge Detection

Segmentation methods that are based on detecting sharp, local changes in intensity.

Edge pixels are pixels at which the intensity of an image changes abruptly, and **edges** (or edge segments) are sets of connected edge pixels). **Edge detectors** are local image processing tools designed to detect edge pixels.

A **line** may be viewed as a thin edge segment in which the intensity of the background on either side of the line is either much higher or much lower than the intensity of the line pixels. lines give rise to so-called “roof edges.”

An **isolated point** may be viewed as a foreground (background) pixel surrounded by background (foreground) pixels.

- Derivative filters can be used to detect point, line or edges of an image.
- First order derivatives generally produce thicker edges.
- The second order derivative has a stronger response to fine details.
- Second order derivatives produce a double-edge response at ramp and step transitions in intensity.
- The sign of the second derivative can be used to determine if the transition from an edge is from dark to light or light to dark.

Detecting Isolated Points

$$\nabla^2 f(x, y) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

$$\nabla^2 f(x, y) = f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)$$

Point detection should be based on the second derivative (Laplacian)

Then, we say that a point has been detected at a location (x,y) on which the kernel is centered if the absolute value of the response of the filter at that point exceeds a specified threshold. Such points are labeled 1 and all others are labeled 0 in the output image, thus producing a binary image.

$$g(x, y) = \begin{cases} 1 & \text{if } |Z(x, y)| > T \\ 0 & \text{otherwise} \end{cases}$$

1	1	1
1	-8	1
1	1	1

Laplacian kernel used for point detection

Detecting Lines

We use second derivatives to result in a stronger filter response, and to produce thinner lines than first derivatives. Thus, we can use the Laplacian kernel. The double-line effect of the second derivative must be handled properly.

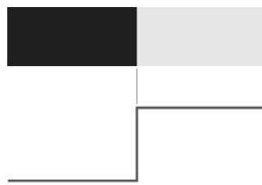
-1	-1	-1	2	-1	-1	-1	2	-1	-1	-1	2
2	2	2	-1	2	-1	-1	2	-1	-1	2	-1
-1	-1	-1	-1	-1	2	-1	2	-1	2	-1	-1
Horizontal			+45°			Vertical			-45°		

Edge Models

Edge detection is an approach used frequently for segmenting images based on abrupt changes in intensity.

Edge models are classified according to their intensity profiles.

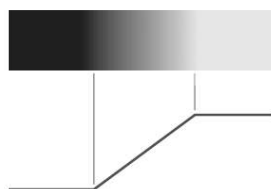
A **step edge** is characterized by a transition between two intensity levels occurring ideally over the distance of one pixel.



Step Edge

These clean, ideal edges can occur over the distance of one pixel, provided that no additional processing (such as smoothing) is used to make them look “real.”

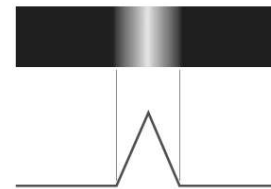
In practice, digital images have edges that are blurred and noisy, with a degree of blurring, edges are more closely modeled as having an intensity **ramp** profile.



Ramp Edge

The slope of the ramp is inversely proportional to the degree to which the edge is blurred. In this model, we no longer have a single “edge point” along the profile. Instead, an edge point now is any point contained in the ramp, and an edge segment would then be a set of such points that are connected.

Roof edges are models of lines through a region, with the base (width) of the edge being determined by the thickness and sharpness of the line. In the limit, when its base is one pixel wide, a roof edge is nothing more than a one-pixel-thick line running through a region in an image.



Roof Edge

We conclude from these observations that the *magnitude* of the first derivative can be used to detect the presence of an edge at a point in an image. Similarly, the *sign* of the second derivative can be used to determine whether an edge pixel lies on the dark or light side of an edge.

The three steps performed typically for edge detection are:

1. **Image smoothing for noise reduction.**
2. **Detection of edge points.** As mentioned earlier, this is a local operation that extracts from an image all points that are potential edge-point candidates.
3. **Edge localization.** The objective of this step is to select from the candidate points only the points that are members of the set of points comprising an edge.

Marr-Hildreth Edge Detector

They suggest that an operator used for edge detection should have two salient features.

1. It should be a differential operator capable of computing a digital approximation of the

first or second derivative at every point in the image.

2. it should be capable of being “tuned” to act at any desired scale, so that large operators can be used to detect blurry edges and small operators to detect sharply focused fine detail.

Marr and Hildreth suggested that the most satisfactory operator fulfilling these conditions is the filter where the Laplacian, and G is the 2-D Gaussian function

$$G(x, y) = e^{-\frac{x^2 + y^2}{2\sigma^2}}$$

$$\begin{aligned}\nabla^2 G(x, y) &= \frac{\partial^2 G(x, y)}{\partial x^2} + \frac{\partial^2 G(x, y)}{\partial y^2} \\ &= \frac{\partial}{\partial x} \left(\frac{-x}{\sigma^2} e^{-\frac{x^2 + y^2}{2\sigma^2}} \right) + \frac{\partial}{\partial y} \left(\frac{-y}{\sigma^2} e^{-\frac{x^2 + y^2}{2\sigma^2}} \right) \\ &= \left(\frac{x^2}{\sigma^4} - \frac{1}{\sigma^2} \right) e^{-\frac{x^2 + y^2}{2\sigma^2}} + \left(\frac{y^2}{\sigma^4} - \frac{1}{\sigma^2} \right) e^{-\frac{x^2 + y^2}{2\sigma^2}}\end{aligned}$$

Collecting terms, we obtain

$$\nabla^2 G(x, y) = \left(\frac{x^2 + y^2 - 2\sigma^2}{\sigma^4} \right) e^{-\frac{x^2 + y^2}{2\sigma^2}}$$

This expression is called the *Laplacian of a Gaussian (LoG)*.

1. Image Smoothing (Gaussian Filtering):

The first step is to blur the image using a **Gaussian low-pass filter**. This reduces noise that could be mistaken for edges.

2. Enhancement (Laplacian Operator)

Next, we apply the **Laplacian operator**, which is a second-order derivative. It highlights regions of rapid intensity change.

3. The LoG Operation

In practice, steps 1 and 2 are combined into a single convolution using the **Laplacian of**

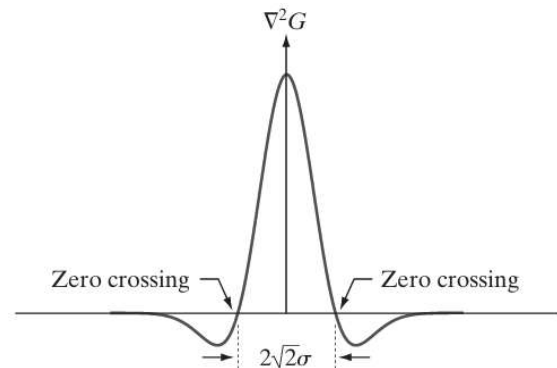
Gaussian (LoG) kernel. This is often called the "**Mexican Hat Operator**" because of its shape.

$$g(x, y) = \nabla^2 [G(x, y) \star f(x, y)]$$

4. Detection (Zero-Crossing)

The actual edges are found by looking for **zero-crossings** in the filtered image.

- An edge exists if the value of a pixel changes sign (e.g., from positive to negative) compared to its neighbors.
- A threshold is usually applied to the variance of the zero-crossing to ignore weak edges caused by noise.



The Canny Edge Detector

Basic Objectives:

1. Low error rate: All edges should be found, and there should be no spurious responses.

2. Edge points should be well localized: The edges located must be as close as possible to the true edges. That is, the distance between a point marked as an edge by the detector and the center of the true edge should be minimum.

3. Single edge point response: The detector should return only one point for each true edge point. That is, the number of local maxima around the true edge should be

minimum. This means that the detector should not identify multiple edge pixels where only a single edge point exists.

1. Noise Reduction (Gaussian Blur): smooth the image using a Gaussian filter.

$$f_s(x, y) = G(x, y) \star f(x, y)$$

We form a smoothed image, $f_s(x, y)$ by convolving f and G

2. Compute the gradient magnitude and angle (direction) images.

$$M_s(x, y) = \|\nabla f_s(x, y)\| = \sqrt{g_x^2(x, y) + g_y^2(x, y)}$$

$$\alpha(x, y) = \tan^{-1} \left[\frac{g_y(x, y)}{g_x(x, y)} \right]$$

with $g_x(x, y) = \partial f_s(x, y) / \partial x$ and $g_y(x, y) = \partial f_s(x, y) / \partial y$.

3. Apply non-maxima suppression to the gradient magnitude image. Analyzes each pixel's gradient magnitude and compares it to its neighbors along the line of the gradient.

Horizontal (0°): Comparing left and right neighbors.

Vertical (90°): Comparing top and bottom neighbors.

Diagonal (45°): Comparing top-right and bottom-left.

Diagonal (135°): Comparing top-left and bottom-right.

If the current pixel's gradient magnitude is **greater** than both of these neighbors, it is a local maximum. It is kept as a potential edge.

If either neighbor has a higher (or equal) magnitude, the current pixel is **suppressed** (its value is set to 0).

4. Threshold $g_N(x, y)$ to reduce false edge points.

If we set the threshold too low, there will still be some false edges (called false positives).

If the threshold is set too high, then valid edge points will be eliminated (false negatives).

This is handled by using *Hysteresis Thresholding*

$$g_{NH}(x, y) = g_N(x, y) \geq T_H$$

$$g_{NL}(x, y) = g_N(x, y) \geq T_L$$

Initially, $g_{NH}(x, y)$ and $g_{NL}(x, y)$ are set to 0.

After thresholding, $g_{NH}(x, y)$ will usually have fewer nonzero pixels than $g_{NL}(x, y)$, but all the nonzero pixels in $g_{NH}(x, y)$ will be contained in $g_{NL}(x, y)$ because the latter image is formed with a lower threshold.

We eliminate from $g_{NL}(x, y)$ all the nonzero pixels from $g_{NH}(x, y)$ by letting:

$$g_{NL}(x, y) = g_{NL}(x, y) - g_{NH}(x, y)$$

The nonzero pixels in $g_{NH}(x, y)$ and $g_{NL}(x, y)$ may be viewed as being “strong” and “weak” edge pixels, respectively. After the thresholding operations, all strong pixels in $g_{NH}(x, y)$ are assumed to be valid edge pixels.

(a) Locate the next unvisited edge pixel, p , in $g_{NH}(x, y)$.

(b) Mark as valid edge pixels all the weak pixels in $g_{NL}(x, y)$ that are connected to p using, say, 8-connectivity.

(c) If all nonzero pixels in $g_{NH}(x, y)$ have been visited go to Step (d). Else, return to Step (a).

(d) Set to zero all pixels in $g_{NL}(x, y)$ that were not marked as valid edge pixels.

At the end of this procedure, the final image output by the Canny algorithm is formed by appending to $g_{NH}(x,y)$ all the nonzero pixels from $g_{NL}(x,y)$.

Thresholding

$$g(x,y) = \begin{cases} 1 & \text{if } f(x,y) > T \\ 0 & \text{if } f(x,y) \leq T \end{cases}$$

When T is a constant applicable over an entire image, the process given in this equation is referred to as *global thresholding*. When the value of T changes over an image, we use the term *variable thresholding*. If T depends on the spatial coordinates (x,y) themselves, then variable thresholding is often referred to as *dynamic or adaptive thresholding*.

$$g(x,y) = \begin{cases} a & \text{if } f(x,y) > T_2 \\ b & \text{if } T_1 < f(x,y) \leq T_2 \\ c & \text{if } f(x,y) \leq T_1 \end{cases}$$

Segmentation by region growing, and by region splitting and merging

Region Growing: As its name implies, region growing is a procedure that groups pixels or subregions into larger regions based on predefined criteria for growth.

The basic approach is to start with a set of "seed" points, and from these grow regions by appending to each seed those neighboring pixels that have predefined properties similar to the seed (such as ranges of intensity or color).

1. Seed Point Selection

The algorithm starts with one or more **seed points**.

- These can be chosen manually by a user (e.g., clicking on a tumor in a medical scan).
- They can be chosen automatically based on specific criteria (e.g., the brightest pixels in an image).

2. Similarity Criterion (The Rule)

You must define a rule that decides if a neighbor is "similar enough" to join the region. Common rules include:

- **Intensity:** The neighbor's gray level is within ± 10 of the seed.
- **Color:** The RGB values are within a certain Euclidean distance.
- **Texture:** The local variance of the neighbor matches the seed.

3. Growing Process

The algorithm examines the **4-connected** (up, down, left, right) or **8-connected** (including diagonals) neighbors of the seed:

- If a neighbor satisfies the similarity criterion, it is added to the region.
- Once a neighbor is added, it becomes a "new seed," and the algorithm checks *its* neighbors.
- This continues recursively or using a queue.

4. Termination

The growth stops when no more neighboring pixels meet the similarity criterion. The final result is a "segment" or a "mask" of the object.

Region Splitting and Merging is an advanced image segmentation technique that overcomes the limitations of simple region growing. While region growing is "bottom-up," this approach is a hybrid that

uses a "top-down" split followed by a "bottom-up" merge.

The Split Phase (Top-Down)

1. **Check Uniformity:** Define a property (like intensity variance). If the variance is higher than a threshold, the region is considered "complex."
2. **Divide:** If the region is complex, split it into **four equal quadrants**.
3. **Recursion:** Repeat this check for each of the four new quadrants.
4. **Stop:** The splitting stops when every block is either uniform or has reached a minimum size

The Merge Phase (Bottom-Up)

After splitting, you often end up with an image that is "over-segmented"—meaning a single object might be broken into many small, adjacent squares that actually belong together.

1. **Neighbor Check:** Look at adjacent blocks that were created during the split phase.
2. **Similarity Test:** If two adjacent blocks are similar enough (e.g., their mean intensities are almost the same), **merge** them into a single region.
3. **Iteration:** Continue merging until no two adjacent regions satisfy the similarity criteria.

The Algorithm Flow: The Quadtree

The logic is most easily visualized as a tree where each node has four children:

- **Root:** The entire image.
- **Nodes:** Regions that needed to be split.
- **Leaves:** The final uniform segments.

Region Segmentation using Clustering and Superpixels

These methods are generally more robust because they don't just look at local neighbors; they look at the statistical personality of the image.

Region Segmentation using Clustering:

Clustering treats segmentation as a **pattern recognition problem**. We don't care where a pixel is (initially); we care about what group its color or intensity belongs to.

K-Means Clustering

This is the most common algorithm used for segmentation.

1. **Feature Space:** Every pixel is mapped into a space based on its features.
2. **Pick K:** You decide how many segments (K) you want.
3. **Assign:** Each pixel is assigned to the nearest cluster center (mean).
4. **Update:** The cluster center is recalculated based on the average of its pixels.
5. **Repeat:** This continues until the centers stop moving.

The Limitation: Standard K-Means ignores *spatial location*. This means pixels of the same color on opposite sides of the image will be put in the same segment, even if they aren't connected.

Region Segmentation using Superpixels

Superpixels are a way to over-segment an image into small, uniform, and meaningful regions that follow the natural boundaries of objects. They act as an intermediate step between raw pixels and final objects.

Why use Superpixels?

- **Complexity Reduction:** An HD image has millions of pixels. Segmenting it into superpixels reduces the computational load.
- **Perceptual Meaning:** Unlike a square pixel, a superpixel respects edges.

SLIC (Simple Linear Iterative Clustering)

The most popular superpixel algorithm. It is essentially **K-Means with a spatial aspect**.

1. **Feature Space:** Instead of just (R,G,B), SLIC uses a 5D space: (R, G, B, x, y).
2. **Spatial Constraint:** By including the (x, y) coordinates, the algorithm forces the clusters to be compact and local.
3. **Grid Initialization:** Seeds are placed in a regular grid across the image.
4. **Local Search:** Pixels only search for the nearest cluster center within a limited $2S \times 2S$ area (where S is the grid interval), making it incredibly fast.